MIT 8103 Advanced Database Systems

MIT 8103 University Examination Management Exercise · Hayatudeen Yusuf · Student ID: 2026/A/MIT/0467301911705

Case study

My case study is MIT 8103 University Examination Management Exercise. I designed the database for examination administrators, lecturers, invigilators, and database students, with separate tables named course, candidate, exam, and registration. It supports these operations: maintain courses and pseudonymous candidate records; schedule course examinations and record venue and seat limits; register candidates for examinations; record attendance and examination scores; query examination rosters and course-level score summaries.

Portfolio 1: Database Design and Modelling

I organised the relational design into 4 tables. Primary keys identify records, foreign keys preserve the links, and checks enforce the stated business rules. A score, when present, must be between 0 and 100 inclusive. The normalisation examples show the sample data at 1NF, 2NF and 3NF. All 13 deliberately invalid inserts were rejected without changing the valid sample rows. Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique key.

Portfolio 2: Query Processing and Optimisation

I ran four SQL queries against the sample records. The join returned 2 rows and included CS101. For the index comparison, I used 10,000 separate performance rows. The median lookup time changed from 0.488 ms to 0.139 ms. Both phases returned the same records. The measured plan used the new index. These timings apply to this lookup and environment.

Portfolio 3: Transactions and Concurrency

I used a temporary counter on registration to test COMMIT, ROLLBACK, and READ COMMITTED isolation. The uncommitted update remained hidden from the other session, and ROLLBACK restored the earlier committed value. Without locking, two increments left the counter at 2. With SELECT FOR UPDATE,

Session B waited for Session A, and the final value was 3. The helper is confined to this exercise; the case's original business fields stay unchanged.

Portfolio 4: NoSQL and Advanced Data Models

I used examination_events for the case's event documents. MongoDB inserted 4 sample events and returned the expected results for 4 queries. Validation rejected an invalid date type and a duplicate event ID. Embedded snapshots keep related details with an event, while the details field can differ by event type. These are fictional events; PostgreSQL continues to enforce its relational rules.

Repository and evidence

https://github.com/Hayatu505/Yusuf_Hayatudeen_2026-A-MIT-0467-301911705

The portfolio folders contain the SQL and JavaScript, explanations, query plans, result tables, and screenshots.

Portfolio 5: Distributed and Cloud Database Exercise

I proposed three MongoDB replica members across separate failure zones. The discussion covers primary elections, majority acknowledgement, secondary lag, and sharding trade-offs for examination_events.

Replication supports availability, while backups address recovery. PostgreSQL needs its own availability plan. This is the permitted discussion option; no live replica set or failover test was deployed.

Individual reflection

I kept course, candidate, exam, and registration in separate tables so their details could be maintained without repeating them in every related record. A score, when present, must be between 0 and 100 inclusive. The constraint tests check this alongside the required values and record links. One issue illustrated by the two-session exercise is that both sessions can read the same value before either update is committed. Locking the row before reading preserves both increments. The index comparison also shows why I need to check the execution plan as well as the final answer: the same result can require different amounts of database work.

AI-use declaration

I used GitHub Copilot to help prepare the database model, scripts, and explanations. The generator ran the PostgreSQL and MongoDB exercises, checked their outputs, and captured the result images. I remain responsible for checking and explaining the submitted work.

References

PostgreSQL: constraints

PostgreSQL: EXPLAIN

PostgreSQL: isolation

PostgreSQL: row locks

MongoDB: data modelling

MongoDB: replication

MongoDB: sharding

Mermaid: ER diagrams